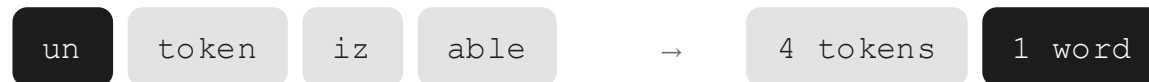


Lecture 5

Tokenization and BPE

Words, tokens, regular expressions, and subword units



Everything downstream counts tokens, not words. This is where that count is decided.

What Lecture 5 covers

Tokenization, end to end

We answer a single question: what is the unit a model reads? We start from words, tokens and types, show why splitting on spaces fails, cover the regular expressions you need, weigh word against character against subword units, and finish with byte-pair encoding worked by hand, plus WordPiece and Unigram.

Three numericals you should be able to reproduce in the exam

- 1 Type-token ratio for a given corpus
- 2 Vocabulary growth under Heaps' law
- 3 A full BPE merge table from a frequency-annotated corpus
- 4 Segmenting a new word with a learned merge table
- 5 Tokenizer fertility for a sentence

The first decision, and the one everything inherits

1 It fixes your vocabulary

Vocabulary size sets the embedding matrix and the output softmax. At 768 dimensions, a 50,000-word vocabulary costs 38.4 million parameters before the model does anything.

2 It fixes your sequence length

Attention costs $O(n^2)$. A tokenizer that splits a sentence into 40 pieces instead of 25 makes every forward pass roughly 2.5 times more expensive.

3 It decides what is out of vocabulary

A word-level tokenizer maps every unseen word to a single UNK symbol. Zipf's law guarantees there will be many.

4 It is not learned end to end

The tokenizer is fitted once, before training, and frozen. A model and a tokenizer from different sources will fail silently and produce plausible nonsense.

`50,000 × 768 = 38,400,000 parameters.    32,000 × 768 = 24,576,000.    The tokenizer chose that difference.`

Words, tokens, types, and the type-token ratio

Corpus: "to be or not to be that is the question"

STEP 1 · COUNT TOKENS

A token is one occurrence. Number them in order:

to(1) be(2) or(3) not(4) to(5) be(6) that(7) is(8)
the(9) question(10)

N = 10 tokens

STEP 2 · COUNT TYPES

A type is one distinct form. Collapse repeats:

{to, be, or, not, that, is, the, question}

V = 8 types ("to" and "be" each occur twice)

STEP 3 · TYPE-TOKEN RATIO

TTR = V / N = 8 / 10 = 0.80

TTR measures lexical variety, but it falls as a corpus grows because common words repeat. Never compare TTR across texts of different lengths without fixing the sample size first, for example by computing it over successive 1,000-token windows.

Vocabulary growth: Heaps' law

$$V = K \cdot N^\beta \quad \text{typically } K \text{ between } 10 \text{ and } 100, \quad \beta \text{ between } 0.4 \text{ and } 0.6$$

WORKED EXAMPLE · TAKE $K = 50$, $\beta = 0.5$

$$N = 1,000,000 \quad V = 50 \cdot (10^6)^{0.5} = 50 \cdot 1,000 = 50,000$$

$$N = 4,000,000 \quad V = 50 \cdot (4 \cdot 10^6)^{0.5} = 50 \cdot 2,000 = 100,000$$

$$N = 16,000,000 \quad V = 50 \cdot (1.6 \cdot 10^7)^{0.5} = 50 \cdot 4,000 = 200,000$$

Quadruple the data, double the vocabulary.

WHY THIS MATTERS

- The vocabulary never stops growing. There is no corpus size at which you have seen every word.
- So a fixed word-level vocabulary will always be missing forms at test time.
- Together with Zipf's law from Lecture 1, this is the whole argument for subword units.

Exam-style question

A corpus of 2 million tokens yields a vocabulary of 70,000 types. Assuming $\beta = 0.5$, estimate K , then predict the vocabulary at 8 million tokens. $K = 70,000 / \sqrt{2 \cdot 10^6} \approx 70,000 / 1,414 \approx 49.5$. At $N = 8 \cdot 10^6$: $V \approx 49.5 \cdot \sqrt{8 \cdot 10^6} \approx 140,000$.

Splitting on whitespace fails immediately

Clitics and contractions

didn't

did + n't, or didn + t, or didn't? Penn Treebank splits it into did and n't.

Multiword expressions

New York

Two whitespace tokens, one semantic unit. So is 'rock and roll'.

Abbreviations

U.S.A.

The final period is part of the token and also possibly the sentence boundary.

Numbers and currency

\$1,200.50

Commas and periods are internal here, not separators.

Hyphenation

state-of-the-art

One token or four? Both conventions are defensible and both are used.

Scripts without spaces

中文分词

Chinese, Japanese and Thai write without word delimiters at all.

Regular expressions: the pieces you need

<code>.</code>	Any character except newline	<code>a.c</code> matches <code>abc</code> , <code>a7c</code>
<code>[abc]</code> <code>[^abc]</code>	Character class; negated class	<code>[aeiou]</code> matches one vowel
<code>[a-z]</code> <code>\w</code> <code>\d</code> <code>\s</code>	Range; word char; digit; whitespace	<code>\w</code> is <code>[A-Za-z0-9_]</code>
<code>*</code> <code>+</code> <code>?</code>	Zero or more; one or more; optional	<code>colou?r</code> matches <code>color</code> , <code>colour</code>
<code>{n}</code> <code>{n,m}</code>	Exactly <code>n</code> ; between <code>n</code> and <code>m</code>	<code>\d{4}</code> matches a four-digit year
<code>^</code> <code>\$</code> <code>\b</code>	Start; end; word boundary	<code>\bcat\b</code> will not match concatenate
<code>(...)</code> <code>(?:...)</code>	Capturing group; non-capturing group	<code>(ab)+</code> matches <code>abab</code>
<code> </code>	Alternation	<code>cat dog</code> matches either

Greedy by default: `*` takes as much as it can. Add `?` to make it lazy: `.*?`

Counting matches: three patterns, one string

Dr. Khan's PhD costs \$1,200.

`\w+`

Dr Khan s PhD costs 1 200

7 matches

Breaks on every non-word character, so the apostrophe splits Khan's and the comma splits the number.

`\S+`

Dr. Khan's PhD costs \$1,200.

5 matches

Whitespace splitting. Keeps the number intact but glues punctuation to the words.

`[A-Za-z]+(?:'[a-z]+)?`

Dr Khan's PhD costs

4 matches

Letters with an optional clitic. Keeps Khan's together and discards the number entirely.

Same input, three defensible tokenizations, three different vocabulary counts. This is why the tokenizer must be recorded alongside every result you report.

Three granularities, and the arithmetic of each

Word-level

Vocabulary: 50,000+

Embeddings at d=768: 38.4M params

Sentence of 20 words: 20 tokens

OOV rate: non-zero, always

Short sequences, huge vocabulary, and an UNK problem Zipf guarantees.

Character-level

Vocabulary: ~100

Embeddings at d=768: 76,800 params

Sentence of 20 words: ~110 tokens

OOV rate: zero

No OOV ever, tiny vocabulary, but sequences 5x longer and each unit carries little meaning.

Subword-level

Vocabulary: 32,000

Embeddings at d=768: 24.6M params

Sentence of 20 words: ~26 tokens

OOV rate: zero

The standard since 2016. Bounded vocabulary with open-vocabulary coverage.

Subword saves 13.8M parameters against word-level and 84 positions of sequence length against character-level.

BPE: merge the most frequent adjacent pair, repeat

TRAINING

1. Split the corpus into words; count word frequencies
2. Split each word into characters; append an end-of-word marker _
3. Vocabulary $\leftarrow$ the set of characters
4. Repeat k times:
 - count every adjacent symbol pair, weighted by word frequency
 - merge the most frequent pair into one new symbol
 - add the new symbol to the vocabulary
5. Return the vocabulary and the ordered merge list

THREE THINGS TO NOTICE

- k, the number of merges, is the only hyperparameter. Final vocabulary size is (number of distinct characters) + k.
- The merge list is ordered, and that order is the model. Applying merges out of order gives a different segmentation.
- Training is done once on a corpus. After that the tokenizer is frozen and purely deterministic.

The corpus we will use for the worked example

5 × l o w _

2 × l o w e s t _

6 × n e w e r _

3 × w i d e r _

2 × n e w _

Initial vocabulary (11 symbols): _ d e i l n o r s t w

Counting every adjacent pair

CONTRIBUTION OF EACH WORD TYPE

5 × (l o w _)	→	(l, o)=5	(o, w)=5	(w, _)=5			
2 × (l o w e s t _)	→	(l, o)=2	(o, w)=2	(w, e)=2	(e, s)=2	(s, t)=2	(t, _)=2
6 × (n e w e r _)	→	(n, e)=6	(e, w)=6	(w, e)=6	(e, r)=6	(r, _)=6	
3 × (w i d e r _)	→	(w, i)=3	(i, d)=3	(d, e)=3	(e, r)=3	(r, _)=3	
2 × (n e w _)	→	(n, e)=2	(e, w)=2	(w, _)=2			

TOTALS, SORTED

(e, r) 9	(r, _) 9	(n, e) 8	(e, w) 8	(w, e) 8	(l, o) 7	(o, w) 7
(w, _) 7	(w, i) 3	(i, d) 3	(d, e) 3	(e, s) 2	(s, t) 2	(t, _) 2

Two pairs tie at 9. Implementations break ties deterministically, usually by the order in which the pair was first seen. We follow the textbook and merge (e,r) first. Note that (w,e) scores 8 and not 10: 'n e w _' contains (e,w) but not (w,e), because nothing follows the w except the end marker.

Running the merges

1 **(e, r) → er**
count 9

5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

New top pair: (er,_) at 6+3 = 9

2 **(er, _) → er_**
count 9

5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

New top pair: (n,e) at 6+2 = 8, tying with (e,w)

3 **(n, e) → ne**
count 8

5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

(e,w) has disappeared; (ne,w) now scores 6+2 = 8

4 **(ne, w) → new**
count 8

5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

Top pairs now (l,o) and (o,w), both at 5+2 = 7

Finishing the merge table

5 (l, o) → lo count 7 5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

6 (l o, w) → low count 7 5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

7 (n e w, e r _) → newer_ count 6 5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

8 (l o w, _) → low_ count 5 5 l o w _ | 2 l o w e s t _ | 6 n e w e r _ | 3 w i d e r _ | 2 n e w _

Result after k = 8 merges

Merge list (ordered): e r · e r _ · n e · n e w · l o · l o w · n e w e r _ · l o w _

Vocabulary size = 11 characters + 8 merges = 19 symbols

Using the merge table to segment new words

Encoding rule: split into characters, then apply every merge in the learned order. Skip a merge if its pair is absent.

CASE A · A WORD SEEN IN TRAINING: LOWER

start	l o w e r _
1 er	l o w er _
2 er_	l o w er_
3 ne	— no match
4 new	— no match
5 lo	lo w er_
6 low	low er_
7 newer_	— no match
8 low_	— no match

Result: low + er_ 2 tokens

A clean morphological split the algorithm was never told about.

CASE B · A WORD NEVER SEEN: NEWEST

start	n e w e s t _
1 er	— no match
2 er_	— no match
3 ne	ne w e s t _
4 new	new e s t _
5 lo	— no match
6 low	— no match
7 newer_	— no match
8 low_	— no match

Result: new + e + s + t + _ 5 tokens

Ugly, but representable. There is no UNK. This is the whole point of BPE.

WordPiece and Unigram: two other ways to choose

BPE

GPT-2, RoBERTa, Llama

Merge the pair with the highest raw frequency.

$$\text{score}(a,b) = \text{count}(ab)$$

Simple and fast. Frequency alone can merge pairs that are common only because their parts are common.

WordPiece

BERT, DistilBERT, ELECTRA

Merge the pair that most increases the likelihood of the training data.

$$\text{score}(a,b) = \frac{\text{count}(ab)}{\text{count}(a) \cdot \text{count}(b)}$$

Normalising by the parts prefers pairs that genuinely co-occur. Marks continuations with ## as in un ##aff ##able.

Unigram / SentencePiece

T5, mT5, ALBERT, XLNet

Start from a large candidate set and prune the pieces that cost least likelihood.

$$\text{maximise } \sum \log P(\text{segmentation})$$

Subtractive rather than additive. Treats the space as an ordinary character, so it is losslessly reversible and script-agnostic.

Tokenizer fertility: how hard your language is to tokenize

$$\text{fertility} = (\text{number of subword tokens}) / (\text{number of whitespace words})$$

WORKED EXAMPLE

Sentence: "The unhappiness was unmistakable"

words = 4

The un happi ness was un mistak able

tokens = 8

fertility = 8 / 4 = 2.0

Why it is not a neutral number

- Tokenizers are fitted on corpora dominated by English, so English fertility is low by construction
- High fertility shortens the effective context window for the same content
- Cost per API call is charged per token, not per word

Lab exercise

Take any parallel corpus with English and Urdu. Run one tokenizer over both sides, divide token count by word count for each, and report the two fertilities with the ratio between them. Measure it rather than quoting a number, because the answer depends entirely on which tokenizer and which corpus.

Hands on: tokenizing the same string four ways

REGEX AND NLTK

```
import re
from nltk.tokenize import word_tokenize
t = "Dr. Khan didn't pay $1,200."

re.findall(r"\w+", t)
['Dr', 'Khan', 'didn', 't', 'pay', '1', '200']

word_tokenize(t)
['Dr.', 'Khan', 'did', "n't", 'pay', '$', '1,200', '.']
```

SPACY

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp(t)

[tok.text for tok in doc]
['Dr.', 'Khan', 'did', "n't", 'pay', '$', '1,200', '.']

[(x.text, x.idx) for x in doc][:3] # offsets
[('Dr.', 0), ('Khan', 4), ('did', 9)]
```

SUBWORD TOKENIZERS FROM HUGGING FACE

```
from transformers import AutoTokenizer
bert = AutoTokenizer.from_pretrained("bert-base-uncased") # WordPiece

bert.tokenize("unaffable")
['un', '##aff', '##able'] # ## marks a continuation piece

# fertility in three lines
words, pieces = t.split(), bert.tokenize(t)
len(pieces) / len(words)
```

What to take away, and what comes next

1 Tokenization fixes your vocabulary, sequence length and cost

Every number you report downstream is conditional on it, so record it.

2 Whitespace splitting fails immediately

Clitics, numbers, abbreviations and space-free scripts all break it. There is only a convention, and it must match at train and test.

3 Subword units solve the open-vocabulary problem

Bounded vocabulary, no UNK, and morphology emerges for free where morphology is frequent.

4 BPE is: merge the most frequent pair, k times

The ordered merge list is the model. Vocabulary size is characters plus k. Rare words fragment but are never lost.

5 WordPiece and Unigram are variations on the same idea

WordPiece normalises by the parts; Unigram prunes rather than merges and is script-agnostic, which suits Urdu.

Next: Lecture 6, Cleaning and Normalization. Unicode, case folding, stop words, stemming, lemmatization, and when to do none of it.
Reading: Jurafsky & Martin, Chapter 2 (regular expressions, text normalization), and Sennrich et al. (2016) on subword units.